Apache Arrow is a development platform for *in-memory analytics*. It contains a set of technologies that enable big data systems to process and move data fast. It specifies a *standardized language-independent columnar memory format* for **flat and hierarchical data**, organized for efficient analytic operations on modern hardware.

****

It consists of a collection of libraries related to in-memory data processing, including specifications for memory layouts and protocols for sharing and efficiently transporting data between systems and processes. When we're talking about in-memory data processing, we're talking exclusively about ***processing data in RAM*** and ***eliminating slow data access*** (as well as redundantly copying and converting data) wherever possible to improve performance. This is where Arrow excels and provides libraries to support this with utilities for streaming and transportation to speed up data access.

****

The primary goal of Arrow is to become the lingua franca of data analytics and processing — the One Format to Rule Them All, so to speak. Different databases, programming languages, and libraries tend to implement and use separate internal formats for managing data, which means that any time you're moving data between these components for different uses, you're paying a cost to *serialize and deserialize that data every time*. Not only that but lots of time and resources get spent reimplementing common algorithms and processing in those different data formats over and over. If we can standardize on an *efficient*, *feature-rich* internal data format that can be widely adopted and used instead, this excess computation and development time is no longer necessary.

****

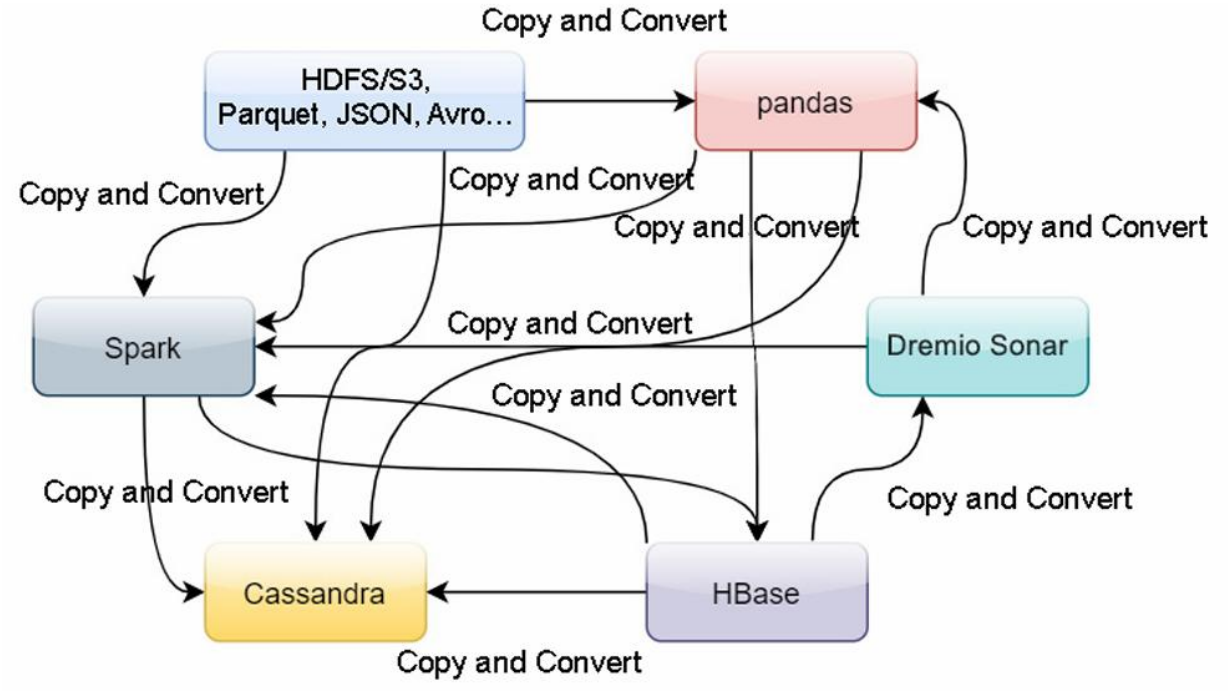

Figure 1.1 – Copy and convert components

****

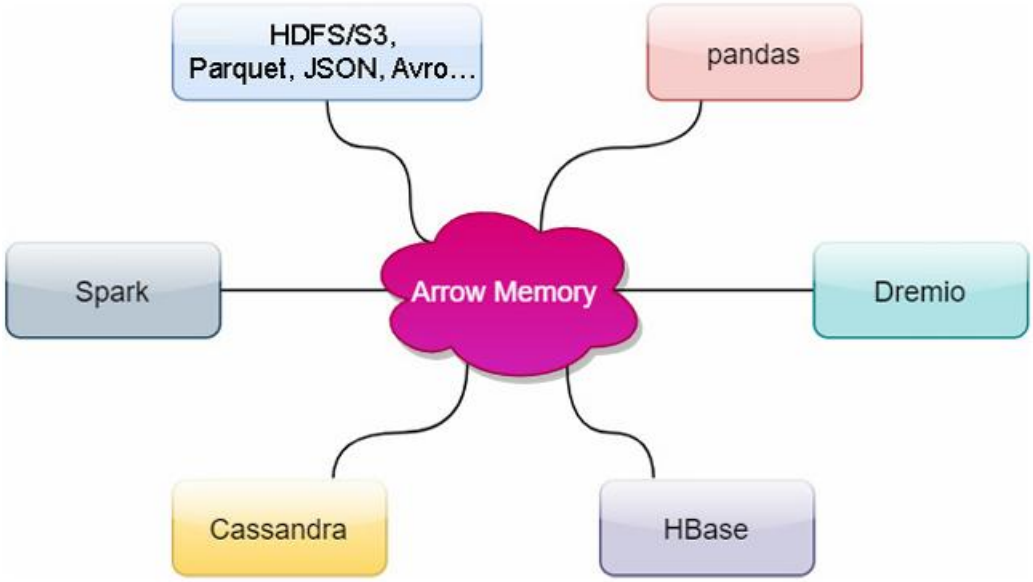

Figure 1.2 – Sharing Arrow memory between components

****

Arrow Database Connectivity (ADBC)

****

Most data processing systems now use distributed processing by breaking the data into chunks and sending those chunks across the network to various workers. So, even if we can share memory across processes on a box, there's still the cost to send it across the network. This brings us to the final piece of the puzzle: the format of raw Arrow data on the wire is the same as it is in memory. You can avoid having to deserialize that data before you can use it (skipping a copy) or reference the memory buffers you were operating on to send it across the network without having to serialize it first. Just a bit of metadata sent along with the raw data buffers and interfaces that perform zero copies can be created to achieve performance benefits, by reducing memory usage and improving throughput.

****

• Using the same high-performance internal format across components allows for much more code reuse in libraries instead of the need to reimplement common workflows.

• The Arrow libraries provide mechanisms to directly share memory buffers to reduce copying between processes by using the same internal representation, regardless of the language. This is what's being referred to whenever you see the term zero-copy.

• The wire format is the same as the in-memory format to eliminate serialization and deserialization costs when sending data across networks between components of a system.

****

Use cases where using Arrow as the internal/intermediate data format can be very beneficial:

• SQL execution engines (such as Dremio Sonar, InfluxDB, or Apache DataFusion)

• Data analysis utilities and pipelines (such as pandas or Apache Spark)

• Streaming and message queue systems (such as Apache Kafka or Storm)

• Storage systems and formats (such as Apache Parquet, Cassandra, and Kudu)

****

By using Arrow, data can be shared between processes and tools, with shared memory increasing the tools available to you for building pipelines, regardless of the language you're operating in. You could take data from *Python*, use it in Spark, and then pass it directly to the **Java virtual machine (JVM)** *without paying the cost of copying between them*.

****

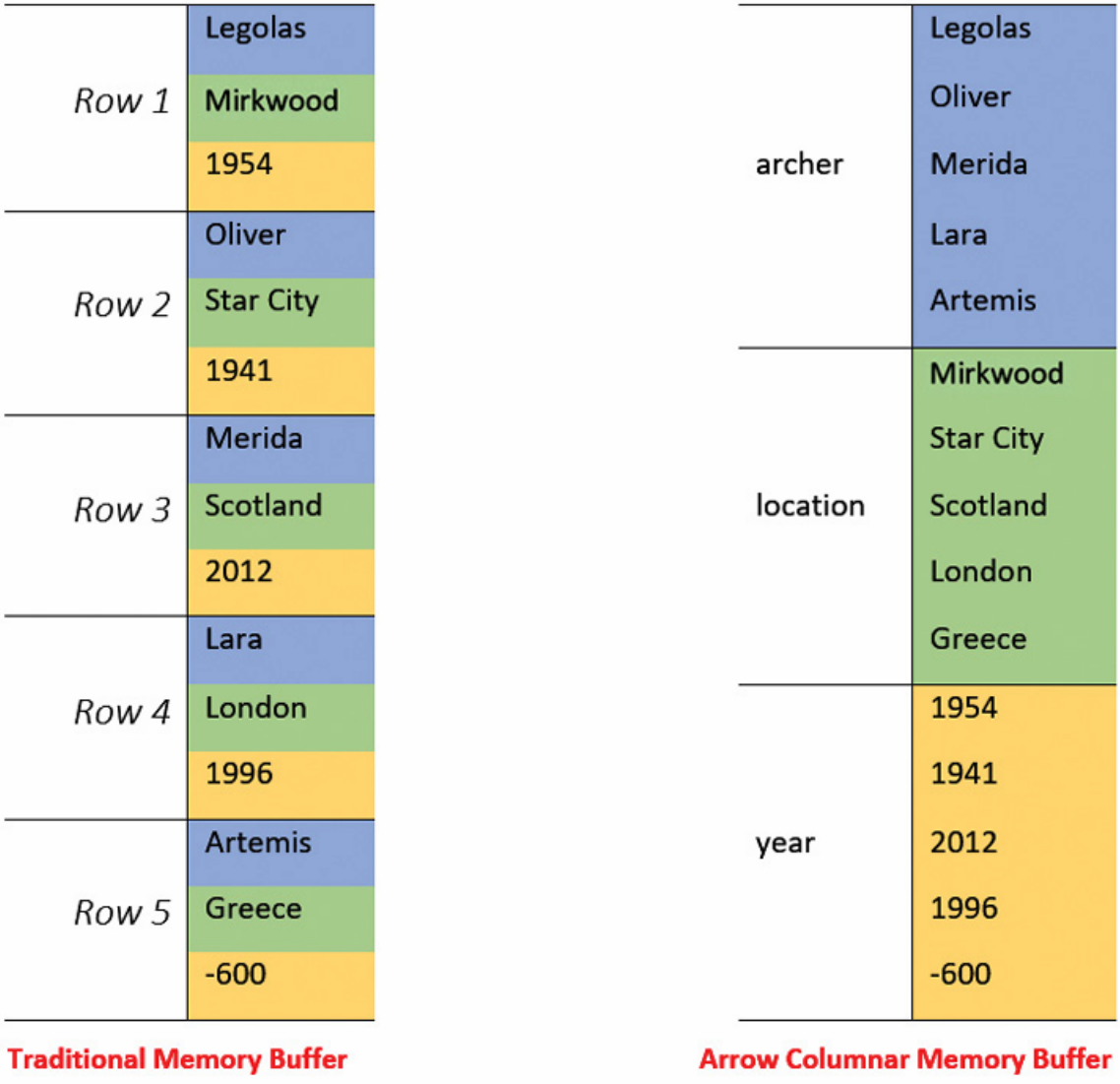

Figure 1.4 – Row versus columnar memory buffers

****

By keeping the column data contiguous in memory, it allows the computations to be vectorized. Most modern processors have *single instruction, multiple data (SIMD) instructions* available that can be taken advantage of for speeding up computations and require the data to be in a contiguous block of memory so that they can operate on it. This concept can be found heavily utilized by graphics cards, and Arrow provides libraries to take advantage of **graphics processing units (GPUs)** precisely because of this. Consider an example where you might want to multiply every element of a list by a static value, such as performing a currency conversion on a column of prices while using an exchange rate

****

Figure 1.5 – SIMD/vectorized versus non-vectorized

****

The Arrow columnar format specification includes definitions of the in-memory data structures, metadata serialization, and protocols for data transportation. The format itself has a few key promises:

• Data adjacency for sequential access

• O(1) (constant time) random access

• SIMD and vectorization-friendly

• Relocatable, allowing for zero-copy access in shared memory

****

Slot: The value in an array identified by a specific index.

****

Buffer/contiguous memory region: A single contiguous block of memory with a given length.

****

Layout Type	Buffer 0	Buffer 1	Buffer 2	Variadic Buffers	Children
Primitive	Bitmap	Data			No
Binary	Bitmap	Offsets	Data		No
Binary View	Bitmap	View headers		Data	No
List	Bitmap	Offsets			1
List View	Bitmap	Offsets	Sizes		1
Fixed-Size List	Bitmap				1
Struct	Bitmap				1 per field
Sparse Union	Type IDs				1 per type
Dense Union	Type IDs	Offsets			1 per type
Null					No
Dictionary Encoded	Bitmap	Data (indices)			Dictionary (not a child)
Run-End Encoded	Bitmap				Run ends and values

Figure 1.6 – Table of physical layouts

****

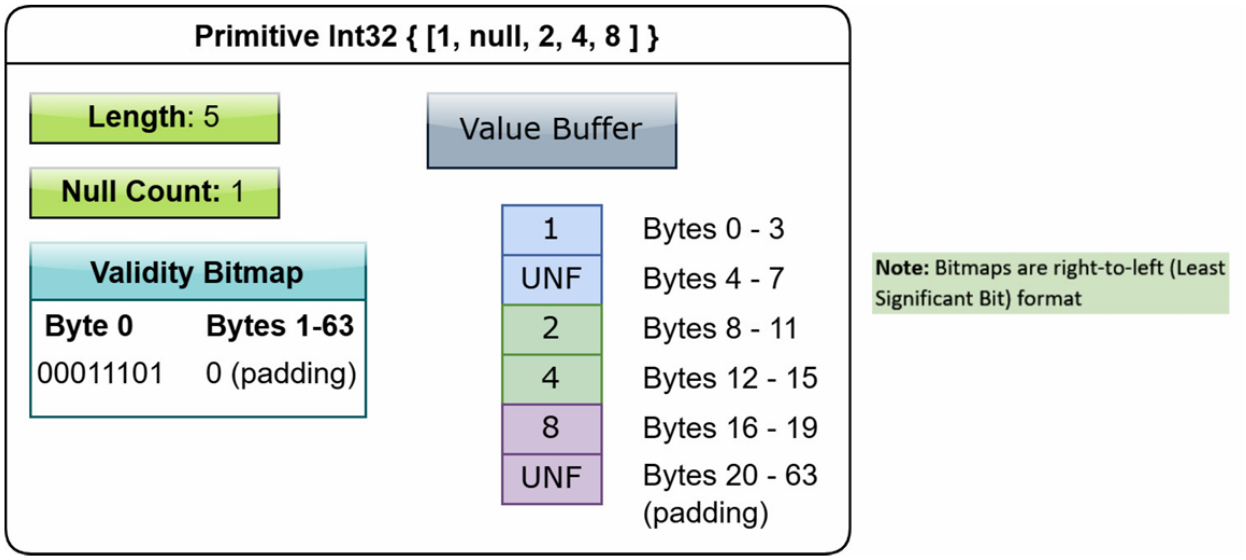

Figure 1.7 – Layout of a primitive int32 array

****

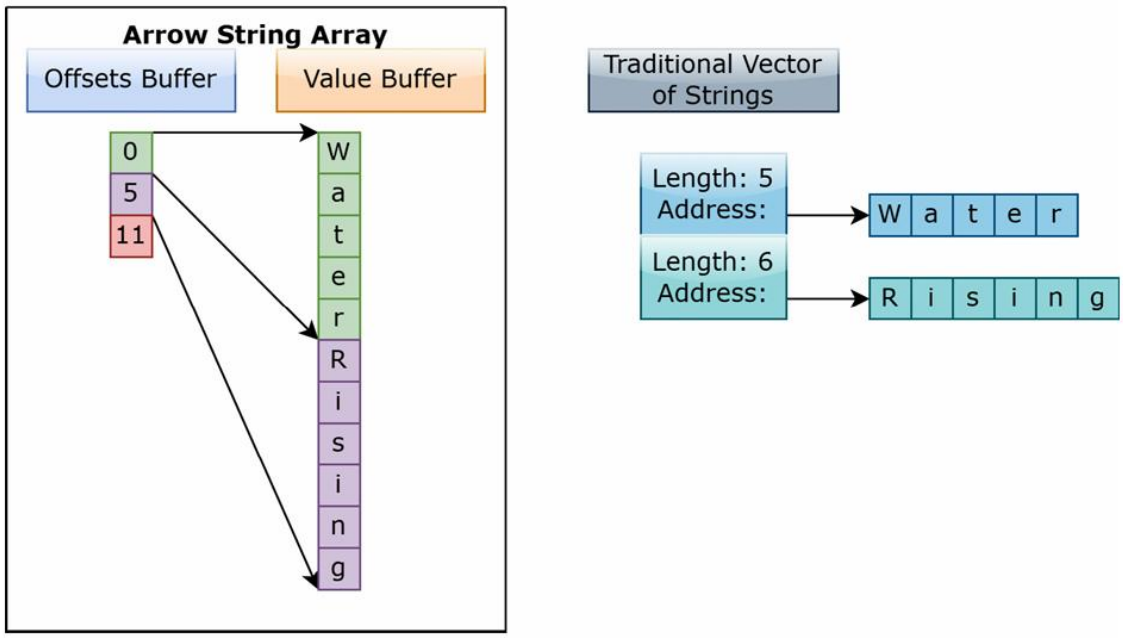

Figure 1.8 – Arrow string versus traditional string vector

****

Instead of the simple approach taken by variable-length binary arrays — using an offsets buffer and data buffer — binary views are a bit more complex. Like most data types, the first buffer is a validity bitmap. As each value in the array consists of 0 or more bytes, the second buffer uses a fixed-size, **16-byte structure** to represent the location and length of each view of bytes (called a **view header**). Finally, while every other data type has a fixed number of buffers, the binary view data type is the first case of a variable number of buffers occurring in an Arrow data type! Let's take a look!

****

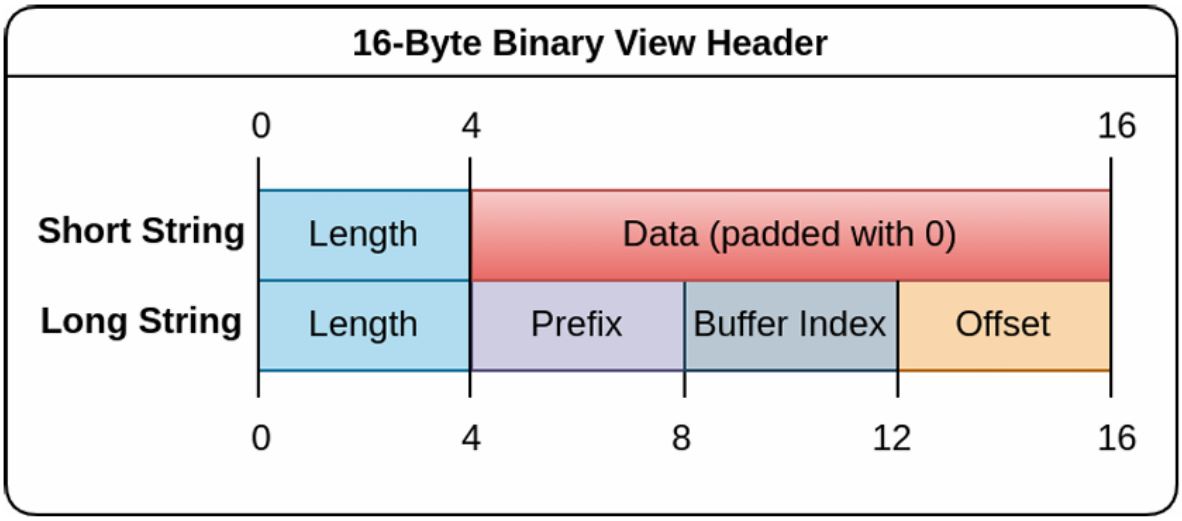

Figure 1.10 – Layout of a binary view header

****

The interpretation of the bytes differs slightly depending on one of two cases: a short string (**length < 12**) or a long string (**length > 12**). If you happen to be familiar with common compiler optimizations, this is inspired by what is commonly referred to as **small string optimization** or **short string optimization**.

****

Important note!

All integers referenced in this layout (the **length**, **buffer index**, and **offset**) must be signed, **32-bit integers**. The reason for this is that some languages – in particular, Java – make **working with unsigned integers** more difficult than with signed integers. So, to ensure easier interoperability within a multi-language environment, the Arrow specification prefers to utilize signed integers where applicable rather than unsigned ones.

****

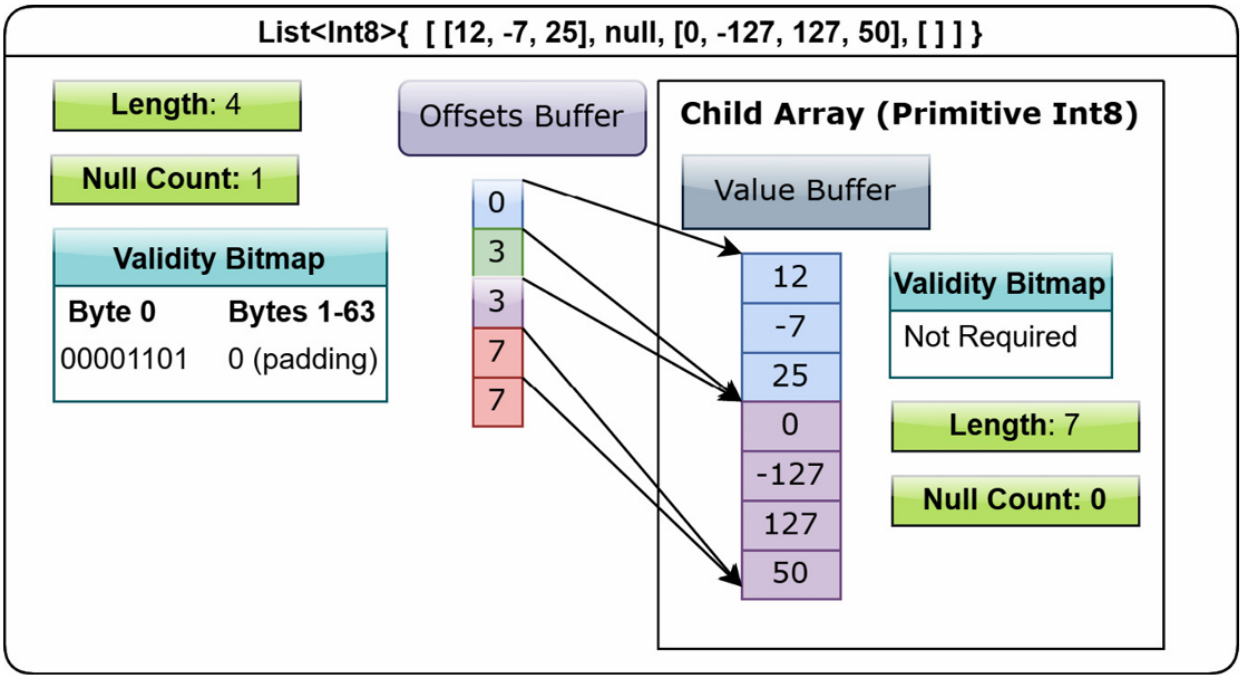

Figure 1.12 – Layout of a list array

****

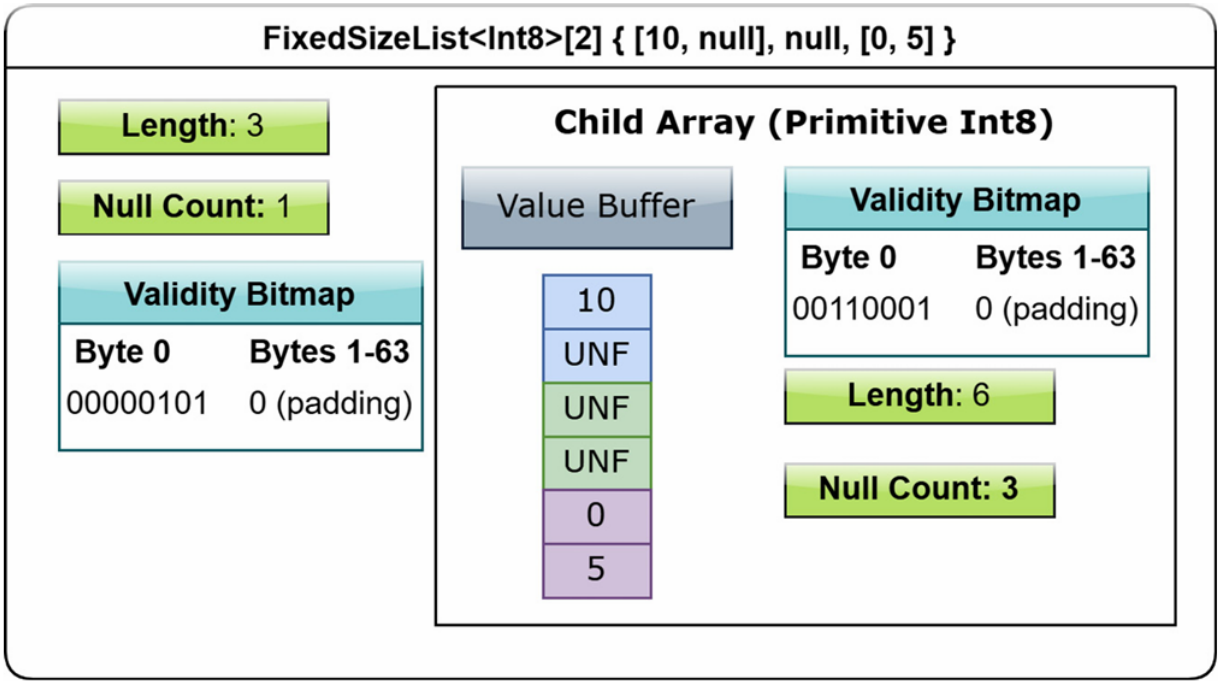

Figure 1.13 – Layout of a fixed-size list array

****